

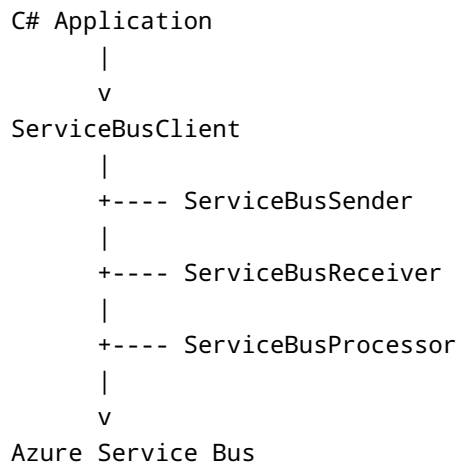
Azure Service Bus – Client, Sender, Receiver and Processor

1. Overview

When working with Azure Service Bus from a C# application, we commonly work with these components:

1. **ServiceBusClient**
2. **ServiceBusSender**
3. **ServiceBusReceiver**
4. **ServiceBusProcessor**

The basic relationship is:



2. ServiceBusClient

What is ServiceBusClient?

`ServiceBusClient` is the main object used to **connect your C# application to Azure Service Bus**.

Simple definition:

ServiceBusClient is the connection between your C# application and Azure Service Bus.

Think of it as the **entry point** to Azure Service Bus.

```
C# Application
  |
  | ServiceBusClient
  v
Azure Service Bus
```

What is required to create ServiceBusClient?

We need the **Service Bus connection string**.

Example:

```
string connectionString = "...";

ServiceBusClient client =
    new ServiceBusClient(connectionString);
```

The connection string contains the information required to authenticate/connect to the Service Bus.

Remember

```
Connection String
  |
  v
ServiceBusClient
  |
  v
Azure Service Bus
```

3. ServiceBusSender

What is ServiceBusSender?

`ServiceBusSender` is used to **send messages** to Azure Service Bus.

Simple definition:

ServiceBusSender is used to send messages to a specific queue or topic.

```
C# Application
  |
  v
ServiceBusSender
  |
  | Send Message
  v
Queue / Topic
```

4. How is ServiceBusSender Created?

A `ServiceBusSender` is created using the `ServiceBusClient`.

We also need to specify the **queue name or topic name**.

Conceptually:

```
ServiceBusClient
  |
  +----- ServiceBusSender
            |
            +----- Queue Name
            OR
            +----- Topic Name
```

Example:

```
ServiceBusClient client =
    new ServiceBusClient(connectionString);

ServiceBusSender sender =
    client.CreateSender("OrderQueue");
```

Here:

- `client` → connects to Azure Service Bus
 - `sender` → sends messages
 - `OrderQueue` → specifies where the message should be sent
-

5. Why Do We Need Queue/Topic Name?

The `ServiceBusClient` connects us to the **Service Bus namespace**.

But a namespace can contain multiple queues and topics.

For example:

```
Service Bus Namespace
|
+---- OrderQueue
|
+---- PaymentQueue
|
+---- NotificationQueue
|
+---- OrderTopic
```

If we want to send a message, we need to tell Azure Service Bus **where the message should go**.

Therefore:

```
ServiceBusClient
|
| CreateSender("OrderQueue")
v
ServiceBusSender
|
v
OrderQueue
```

6. ServiceBusReceiver

What is ServiceBusReceiver?

`ServiceBusReceiver` is used to **receive messages from Azure Service Bus**.

Simple definition:

ServiceBusReceiver is used to receive messages from a queue or from a topic subscription.

```
Queue
|
| Message
v
ServiceBusReceiver
|
v
C# Application
```

7. Creating a Receiver for a Queue

When receiving messages from a **queue**, we provide the queue name.

```
ServiceBusClient
|
v
ServiceBusReceiver
|
+----- Queue Name
```

Example:

```
ServiceBusReceiver receiver =
    client.CreateReceiver("OrderQueue");
```

Here:

```
OrderQueue
|
| Message
v
ServiceBusReceiver
|
v
C# Application
```

8. Creating a Receiver for a Topic

Receiving messages from a topic is slightly different.

A topic normally has one or more **subscriptions**.

Example:

```
OrderTopic
|
+---- PaymentSubscription
|
+---- InventorySubscription
|
+---- NotificationSubscription
```

You don't directly receive a message from the topic.

Instead, you receive the message from a **subscription within the topic**.

Therefore, when creating a receiver for a topic, we need:

1. Topic name
2. Subscription name

Conceptually:

```
ServiceBusClient
|
v
ServiceBusReceiver
|
+---- Topic Name
|
+---- Subscription Name
```

Example:

```
ServiceBusReceiver receiver =
    client.CreateReceiver(
        "OrderTopic",
        "PaymentSubscription");
```

9. Queue vs Topic Receiving

This is an important concept.

Queue

For a queue, we provide:

Queue Name

Example:

```
client.CreateReceiver("OrderQueue");
```

Flow:

```
OrderQueue
  |
  v
Receiver
```

Topic

For a topic, we provide:

Topic Name + Subscription Name

Example:

```
client.CreateReceiver(
    "OrderTopic",
    "PaymentSubscription");
```

Flow:

```

OrderTopic
  |
  v
PaymentSubscription
  |
  v
Receiver

```

Easy way to remember

```

Queue
  ↓
Queue Name

Topic
  ↓
Topic Name + Subscription Name

```

10. Why Do Topics Use Subscriptions?

Suppose we have an `OrderTopic`.

```

          OrderTopic
          |
+-----+-----+
|         |         |
|         v         v         v
Payment   Inventory   Notification
Subscription Subscription Subscription

```

The same published message can be made available to multiple subscriptions.

For example:

```

Order Created
  |
  v
OrderTopic
  |
+---- PaymentSubscription

```



```
|
+---- InventorySubscription
|
+---- NotificationSubscription
```

Each subscription can be consumed independently.

Therefore, when a consumer wants to receive messages from a topic, it reads from its **subscription**.

11. ServiceBusProcessor

What is ServiceBusProcessor?

`ServiceBusProcessor` can be thought of as an **advanced message receiver**.

Simple definition:

ServiceBusProcessor receives messages and provides additional features for continuously processing incoming messages.

The key difference is that with a processor, we can register a function/delegate that is automatically executed when a new message arrives.

Think of it as:

```
ServiceBusProcessor
    =
Receiver
+
Message Processing Features
```

12. How ServiceBusProcessor Works

Suppose we have:

```
OrderQueue
|
| New Message
v
```

```
ServiceBusProcessor
  |
  v
Message Handler
  |
  v
Process Order
```

We register a message-handling function.

Conceptually:

```
processor.ProcessMessageAsync += MessageHandler;
```

Whenever a message arrives, the handler is invoked.

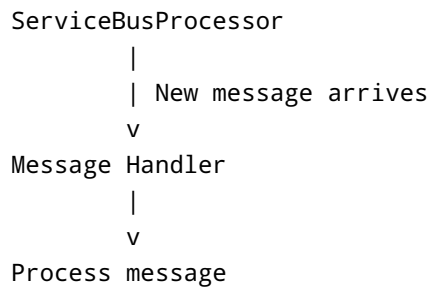
```
New Message Arrives
  |
  v
ServiceBusProcessor
  |
  v
Message Handler Executes
  |
  v
Process Message
```

13. Why Use ServiceBusProcessor?

Without a processor, you may explicitly receive messages:

```
Receive Message
  |
  v
Process Message
  |
  v
Receive Next Message
```

With a processor, you can register a handler:



The processor handles much of the message-receiving workflow for you.

14. Simple Example of ServiceBusProcessor

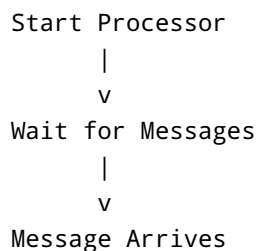
Conceptually:

```
ServiceBusProcessor processor =  
    client.CreateProcessor("OrderQueue");  
  
processor.ProcessMessageAsync += MessageHandler;  
processor.ProcessErrorAsync += ErrorHandler;  
  
await processor.StartProcessingAsync();
```

Message handler:

```
async Task MessageHandler(ProcessMessageEventArgs args)  
{  
    string message = args.Message.Body.ToString();  
  
    Console.WriteLine(message);  
}
```

The important idea is:



|
v
Message Handler Executes

15. Receiver vs Processor

This is an important interview question.

ServiceBusReceiver	ServiceBusProcessor
Used to receive messages	Used to receive and process messages continuously
You have more direct control over receiving	Provides an event/handler-based processing model
Application explicitly receives messages	Processor invokes handlers when messages arrive
Suitable for explicit receive operations	Suitable for continuous/background message processing

Easy way to remember

Receiver:

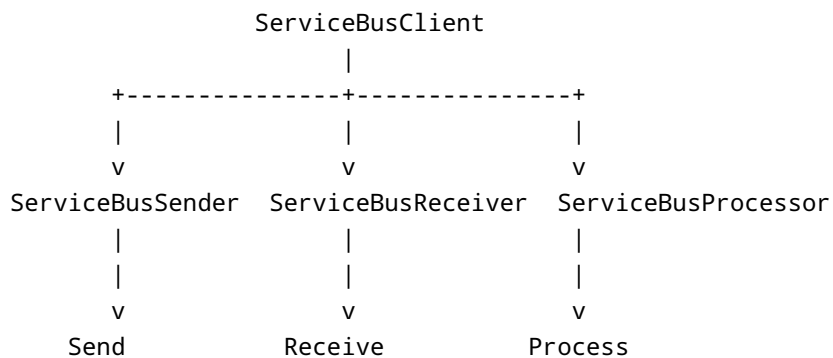
"Give me a message."

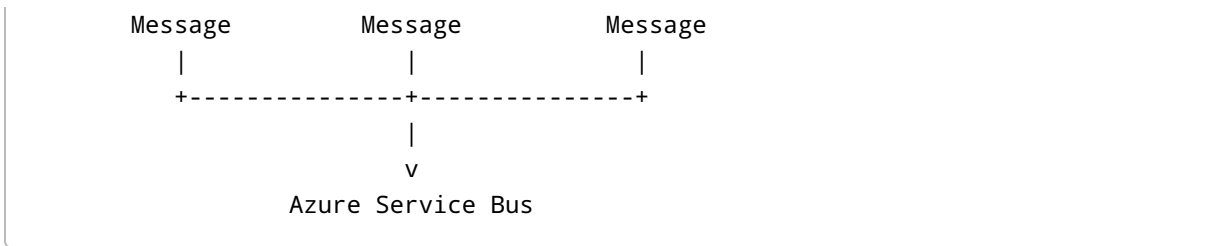
Processor:

"Keep listening and call my function whenever a message arrives."

16. Complete Relationship

The four important objects can be understood like this:





17. Real-Time Example

Consider an e-commerce application.

We have:

```
Order Service
Email Service
Payment Service
```

Sending an Order Message

The Order Service creates a `ServiceBusClient`.

```
Order Service
  |
  v
ServiceBusClient
  |
  v
ServiceBusSender
  |
  v
OrderQueue
```

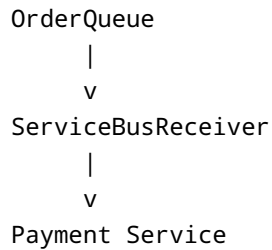
The sender sends:

```
Order Created
```

message to the queue.

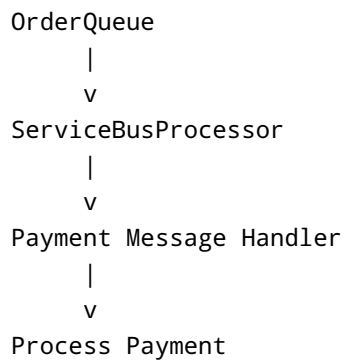
Receiving the Message

The Payment Service can create a receiver:



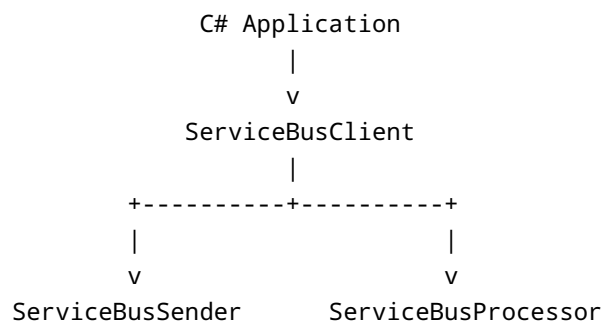
Processing Continuously

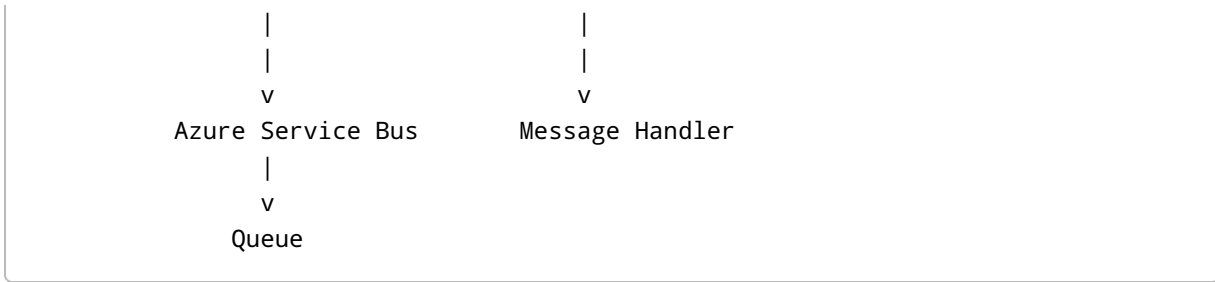
Alternatively, the Payment Service can use a processor:



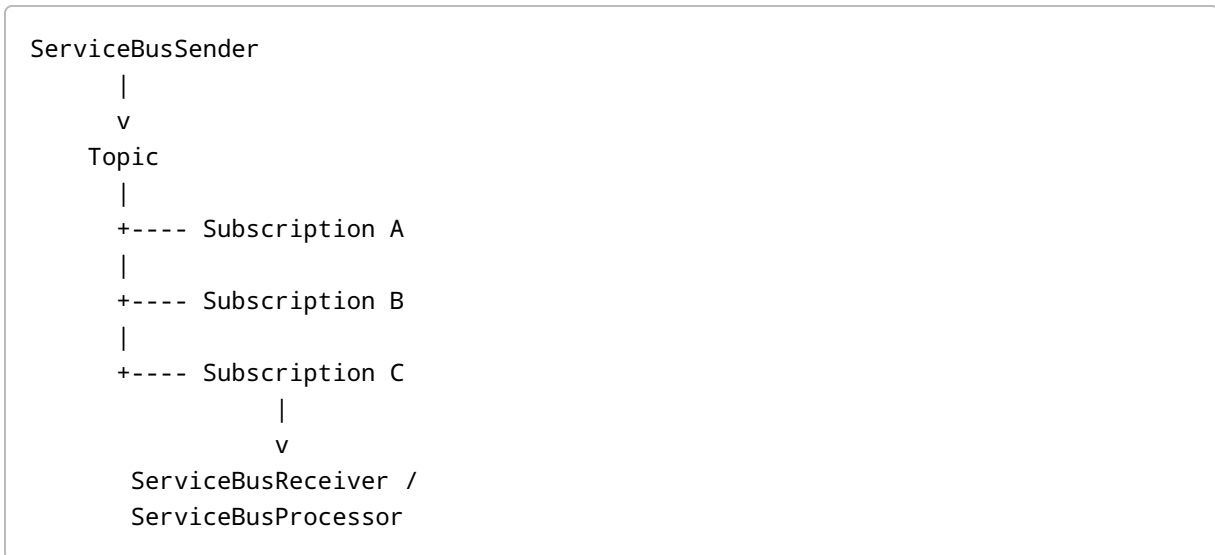
Whenever a new order message arrives, the processor invokes the message handler.

18. End-to-End Flow





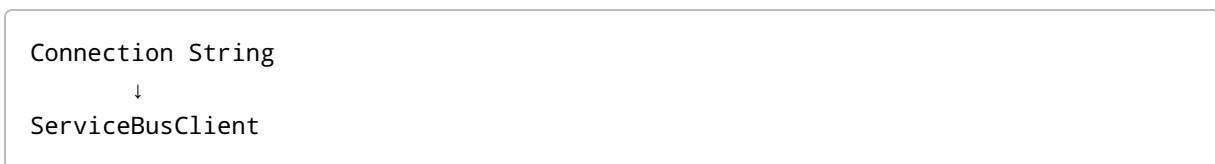
For a topic:



19. Key Differences

ServiceBusClient

Purpose: Connect C# application to Azure Service Bus.



ServiceBusSender

Purpose: Send messages.

```
ServiceBusClient
  ↓
ServiceBusSender
  ↓
Queue / Topic
```

ServiceBusReceiver

Purpose: Receive messages.

```
Queue
  ↓
ServiceBusReceiver
  ↓
Application
```

For a topic:

```
Topic
  ↓
Subscription
  ↓
ServiceBusReceiver
  ↓
Application
```

ServiceBusProcessor

Purpose: Continuously receive and process messages using handlers.

```
Queue / Subscription
  ↓
ServiceBusProcessor
  ↓
Message Handler
  ↓
Business Logic
```

20. Interview Questions

Q1. What is ServiceBusClient?

Answer:

`ServiceBusClient` is the main client object used by a C# application to connect to Azure Service Bus. It is created using Service Bus connection information and is then used to create senders, receivers, and processors.

Q2. What is ServiceBusSender?

Answer:

`ServiceBusSender` is used to send messages to a specific queue or topic. It is created using `ServiceBusClient`.

Q3. What is ServiceBusReceiver?

Answer:

`ServiceBusReceiver` is used to explicitly receive messages from a queue or from a topic subscription.

Q4. What is ServiceBusProcessor?

Answer:

`ServiceBusProcessor` provides a higher-level message processing model. We register message and error handlers, and the processor continuously receives messages and invokes the appropriate handlers when messages arrive.

Q5. How do you receive a message from a topic?

Answer:

Messages published to a topic are consumed through a subscription. Therefore, we need to specify the topic name and subscription name when creating the receiver or processor.

Topic
↓
Subscription
↓
Receiver / Processor

Q6. What is the difference between Sender and Receiver?

Sender:

Application → Service Bus

Receiver:

Service Bus → Application

Q7. What is the difference between Receiver and Processor?

Receiver:

Used for explicit message receiving.

Processor:

Continuously listens for messages and invokes registered handlers when messages arrive.

21. Quick Revision

Remember these four words:

CLIENT → Connect
SENDER → Send
RECEIVER → Receive
PROCESSOR → Process

Easy Memory Trick

Client = Connection

Sender = Send

Receiver = Receive

Processor = Receive + Automatically Process

22. Final Interview Summary

In a C# application, `ServiceBusClient` is used to connect to Azure Service Bus. From the client, we can create a `ServiceBusSender` to send messages to a queue or topic. We can create a `ServiceBusReceiver` to explicitly receive messages from a queue or topic subscription. Alternatively, we can use `ServiceBusProcessor`, which provides a continuous message-processing model where registered handlers are invoked when messages arrive. For queues we specify the queue name, while for topics we work with a topic and its subscription.